📜 نظام المحاسبة الكامل — توثيق المشروع ورحلة التطوير

Full-Stack Double-Entry Accounting System — University Final Project

> وثيقة شاملة بالعربي تشرح المشروع: التقنيات، البنية، قاعدة البيانات، المفاهيم المحاسبية، الميزات، ورحلة التطوير خطوة بخطوة. مفيدة لعرض التخرّج والمناقشة.

---

١) نبذة عن المشروع

نظام محاسبي إلكتروني متكامل مبني على مبدأ **القيد المزدوج (Double-Entry Bookkeeping)**: كل عملية مالية تُسجَّل في دفتر الأستاذ بجهتين — **جهة مدينة (Debit)** و**جهة دائنة (Credit)** — ومجموع المدين يساوي دائمًا مجموع الدائن.

الوحدات (Screens) في النظام:

- لوحة التحكم (Dashboard)
- دليل الحسابات (Chart of Accounts)
- العملاء (Customers)
- الموردون (Suppliers)
- المنتجات (Products)
- **المشتريات / إدخال المخزون (Purchases)**
- الفواتير والمبيعات (Invoices)
- الدفعات (Payments)
- قيود اليومية / دفتر الأستاذ (Journal)
- التقارير المالية (Reports)
- المستخدمون (Users)
- سجل التدقيق (Audit Log)

---

٢) التقنيات المستخدمة (Tech Stack)

الطبقة	التقنية	السبب
Frontend	**JavaScript** خام **(Vanilla)** HTML + CSS — بدون React/Angular	أخف، أوضح للشرح، ويلبّي متطلبات المشروع
التصميم	**Tailwind CSS** (نظام تصميم "Stitch "ds + Material Symbols + خط Inter	واجهة موحّدة، دعم RTL، وضع فاتح/داكن
Backend	**Node.js + Express**	API بسيط وسريع
قاعدة البيانات	**MySQL** عبر `mysql2`	علائقية، تدعم المعاملات (Transactions) الدقيقة
المصادقة	**JWT** + **bcrypt**	جلسات آمنة + تشفير كلمات السر
الأمان	`helmet` + `cors`	رؤوس حماية + سياسة أصل الطلبات
التحقق	`express-validator`	عدم الثقة بمدخلات العميل
الإعدادات	`dotenv` (.env)	إخفاء بيانات الاتصال والمفاتيح
أثناء التطوير	`nodemon`	إعادة تشغيل تلقائي

مكتبات الـ Backend الفعلية (من `package.json`): `express` ، `mysql2` ، `bcrypt` ، `jsonwebtoken` ، `helmet` ، `cors` ، `dotenv` ، `express-validator` ، وللتطوير `nodemon` .

---

٣) البنية المعمارية (Architecture)

```
[ Frontend  (client/) ]  --HTTP + JSON-->  [ Backend API  (Express :3000) ]  --SQL-->  [ MySQL :3306 ]
   Vanilla JS + fetch                          Routes > Controllers > Services > Models
```

٣.١ طبقات الـ Backend (`server/`)

المجلد	الدور
`server.js`	نقطة البداية: تركيب الـ middleware وتسجيل كل المسالك
`config/`	`db.js` (الاتصال) + `accounting.js` (أكواد الحسابات الافتراضية)
`routes/`	تعريف عناوين الـ API + الصلاحيات المطلوبة لكل مسار
`controllers/`	طبقة HTTP رقيقة: قراءة الطلب، نداء الـ service، ردّ JSON موحّد
`services/`	**عقل المشروع**: المنطق المالي، المعاملات، قواعد العمل
`models/`	كل استعلامات SQL (Parameterized) لكل جدول
`middleware/`	`auth` (JWT + أدوار)، `errorHandler` ، `validate` مركزي
`validators/`	قواعد `express-validator`
`utils/`	`money` (حساب بالسنت الصحيح)، `asyncHandler` ، `response` ، `ApiError` ، `i18n` ، `jwt`

٣.٢ بنية الـ Frontend (`client/`)

كل صفحة HTML تحمّل السكربتات بترتيب ثابت:

```
config → api → auth → ui → layout → crud → lookups → <منطق الصفحة>
```

الملف	الدور
`api.js`	غلاف حول `fetch` : يضيف التوكن + رأس اللغة `X-App-Lang` ، ويوحّد قراءة الرد
`auth.js`	حفظ الجلسة، حراسة الصفحات (`requireAuth`)، تسجيل الخروج
`ui.js`	الترجمة والاتجاه: `isRtl` ، `bi(en,ar,fa)` ، تنسيق المال `$` ، الثيم
`layout.js`	القالب المشترك (القائمة الجانبية + الشريط العلوي) عبر map `NAV`
`crud.js`	أدوات مشتركة: `renderTable` و `openFormModal`
`lookups.js`	محمّلات مُخفّاة للقوائم (عملاء/منتجات/موردون/حسابات)

---

٤) قاعدة البيانات (`database/`)

ملفان: `schema.sql` (الجداول) + `seed.sql` (بيانات البداية). التشغيل عبر `scripts/load-db.ps1` .

أهم الجداول: `chart_of_accounts` · `users` (شجرة الحسابات) · `products` (+ `stock_quantity`) · `customers` · `suppliers` · `journal_entries` + `journal_lines` · `audit_logs` · `purchases` + `purchase_items` · `invoices` + `invoice_items` .

قواعد تصميم مهمة:

- **المبالغ** `DECIMAL(15,2)` — ممنوع FLOAT/DOUBLE (دقة محاسبية).
- **مفاتيح أجنبية (FK)** مع قواعد حذف محسوبة:
  - `CASCADE` للبنود (invoice_items/purchase_items بلا أبوها ما بتعنشي شي).
  - `RESTRICT` لحماية السجل المالي (ما ينحذف عميل عنده فواتير).
  - `SET NULL` لربط الحسابات.
- **حذف ناعم (Soft Delete)** عبر `is_active` بدل الحذف الفعلي.
- **عمود** `products.inventory_account_id` يفرّق **البضاعة** (لها حساب مخزون → تُتابَع بالكمية) عن **الخدمة** (NULL → تُباع بلا مخزون).

---

٥) المفاهيم المحاسبية والتقنية الأساسية

1. **القيد المزدوج** (`journalService`): القيد لا يمرّ إلا إذا: ≥ ٢ سطر، كل سطر مدين أو دائن (لا الاثنين)، **مجموع المدين = مجموع الدائن**، وكل حساب موجود وورقي (leaf).
2. **الدقة المالية** (`money.js`): كل الحسابات **بأعداد صحيحة (سنتات)** لتفادي أخطاء الفواصل العائمة (`0.1+0.2`)، والتحويل لـ"123.45" فقط عند العرض/التخزين.
3. **المعاملات الذرّية (Transactions):** الفاتورة + قيدها + حركة المخزون داخل `transaction` واحد (`beginTransaction/commit/rollback`) مع أقفال `SELECT ... FOR UPDATE` — إمّا تنجح كلها أو ترتد كلها.
4. **الصلاحيات (RBAC):** `viewer` / `accountant` / `admin` . القراءات لأي مستخدم مسجّل، الكتابة على البيانات لـ admin+accountant، إدارة المستخدمين والتدقيق لـ admin فقط.
5. **سجل التدقيق (Audit):** كل عملية كتابة تُسجَّل (`auditService`).
6. **توحيد الاستجابة:** كل الردود JSON بنفس الشكل (`success/message/data/meta` ، والأخطاء `errors[]`).
7. **تعدد اللغات والاتجاه:** ثلاث لغات (EN/AR/FA) مع RTL، وأي عبارة غير مترجمة ترجع للإنكليزي تلقائيًا. العملة **دولار $** موحّدة على كل النظام.

---

٦) دورة العمل المالية (مثال سوبرماركت)

1. **شرائي من مورّد** (شاشة المشتريات) ← تزيد الكمية + قيد: `مدين المخزون / دائن ذمم المورد` .
2. **فاتورة مبيعات** (مسودة ← إصدار) ← عند الإصدار: `مدين ذمم العميل / دائن الإيرادات` + **تنقص الكمية** + `مدين تكلفة البضاعة المباعة (COGS) / دائن المخزون` .
3. **دفعة من عميل** ← `مدين الصندوق/البنك / دائن ذمم العميل` .
4. **التقارير** (ميزان مراجعة، قائمة دخل، مركز مالي) ← تتولّد من القيود.

> **مسودة vs مصدّرة:** المسودة ما بتأثر بدفتر الأستاذ ولا بالمخزون. التحويل لـ"مصدّرة" هو اللي بيقيد وبينقص الكمية.

---

٧) رحلة التطوير — شو عملنا خطوة بخطوة

> ملاحظة: المحادثة الأصلية كانت طويلة، فالتطبيق لخّص أقدمها. هذا ترتيب أهم المراحل يلي مرّينا فيها:

المرحلة ١ — الأساس Full-Stack

- بناء هيكل المشروع (client/server/database)، الاتصال بـ MySQL، الـ middleware، الأخطاء الموحّدة.
- مصادقة bcrypt + JWT + أدوار (RBAC) + سجل تدقيق.
- CRUD كامل: دليل الحسابات، العملاء، الموردون، المنتجات، الفواتير، الدفعات، القيود، التقارير، المستخدمون.
- دقة مالية بالسنتات + قيد مزدوج متوازن.

المرحلة ٢ — التوطين واللغات

- **إضافة الفارسي (FA)** كلغة ثالثة RTL مع fallback للإنكليزي.
- **توحيد العملة على الدولار $** على كل الشاشات (منتجات/حسابات/داشبورد/فواتير/دفعات) — تغيير رمز/تسمية فقط بلا أسعار صرف.
- توحيد اتجاه الواجهة (RTL/LTR) بمنطق CSS لا يتفرّع لكل لغة.

المرحلة ٣ — إصلاح حسابات الذمم (قرار جذري)

- المشكلة: عميل جديد من الواجهة بلا `account_id` ← الفاتورة تفشل ("Customer has no receivable account").
- الحل: **إنشاء حساب فرعي تلقائي** لكل عميل (ذمم مدينة AR تحت 1300) وكل مورد (ذمم دائنة AP تحت 2100) داخل نفس المعاملة، + سكربت `backfillPartyAccounts.js` للأقدم.

المرحلة ٤ — تقييد طرق الدفع

- الدفعات صارت **نقدًا (Cash) وبطاقة (Card)** فقط (شيلنا بنك/شيك).

المرحلة ٥ — دورة حياة الفاتورة

- إضافة **إصدار (Issue)** للمسودة (بينقلها لمصدّرة + يقيد + ينقص المخزون) و**حذف (Delete)** للمسودة فقط.
- حماية: الفاتورة المصدّرة ما تنحذف (لازم تُلغى)، والمسودة ما تنقيد.

المرحلة ٦ — المخزون (المستوى الكامل B)

- عمود `stock_quantity` + جداول `purchases` / `purchase_items` + وحدة مشتريات كاملة (API + شاشة).
- الشرائي يزيد الكمية ويقيد `AP مدين المخزون / دائن` ويحدّث تكلفة المنتج (last-cost).

- البيع ينقص الكمية ويقيد `مدين COGS / دائن المخزون` ، و**يمنع البيع أكبر من المتوفر**.
- ملف (idempotent) `database/migrations/001_inventory.sql` للقواعد الموجودة.

المرحلة ٧ — إصلاح الخدمات vs البضاعة

- الخطأ: فاتورة فيها "خدمة تركيب" انرفضت ("Insufficient stock"). السبب: النظام كان بيعامل كل منتج كبضاعة.
- الحل: **المنتج اللي بلا حساب مخزون = خدمة** → تُباع بلا كمية ولا COGS، والشرائي يرفض تخزين خدمة.

المرحلة ٨ — إصلاح توليد الأرقام

- الخطأ القديم: رقم الفاتورة كان `COUNT+1` ← تصادم (409) لما الحذف يسيّب فجوات بالتسلسل.
- الحل: `MAX(الرقم التسلسلي)+1` للفواتير والمشتريات (آمن من الفجوات).

---

٨) قرارات هندسية وأخطاء تعلّمناها

- **الحسابات داخل معاملة واحدة** — لا فاتورة بلا قيدها ولا العكس.
- **السنتات الصحيحة بدل float** — أساس الدقة المحاسبية.
- **الخادم هو مصدر الحقيقة للسعر** — سعر الوحدة بيجي من كتالوج المنتج، مش من العميل.
- **الفرق بين البضاعة والخدمة** عبر `inventory_account_id` (لا خلط بالمخزون).
- **توليد الأرقام على MAX مش COUNT** (الفجوات بتسبّب تصادم).
- **UTF-8 عند استيراد SQL بالعربي** عبر `Start-Process -RedirectStandardInput` (الـ pipe بيحوّل العربية لـ"؟").

---

٩) كيف تشغّل المشروع محليًا

```
# 1) قاعدة البيانات (على XAMPP MySQL 3306:)
npm run db:load          # يشغّل schema.sql ثم seed.sql

# 2) الـ API على 3000:
npm start                # أو: npm run dev (nodemon)

# 3) الواجهة على 5500: (خادم ملفات ثابتة)
npx http-server client -p 5500 -c-1
```

- **الدخول:** `admin / Admin@123` (و `viewer` / `accountant`).
- **البيئة:** ملف `.env` يحوي بيانات الاتصال و `JWT_SECRET` (غير مربوط بـ git).

---

١٠) نصائح لعرض التخرّج

- اشرح **القيد المزدوج** بمثال حيّ: فاتورة ← كيف اتوازن المدين/الدائن.
- وزّي **الدقة المالية** (سنتات، DECIMAL) كنقطة قوة.
- وزّي **RBAC**: نفس النظام بثلاث صلاحيات بواجهات مختلفة.
- وزّي **المخزون**: شرائي يزيد، بيع ينقص، منع العجز، وCOGS.
- وزّي **تعدد اللغات + RTL** كميزة UI متقدّمة.

هذي الوثيقة مولّدة من سجل محادثة التطوير. النسخة الخام من سجل الجلسة في `docs/chat-history/` .